

scitech-librarian — Manual del usuario

versión 3.4.2

2026-09-04

Índice

1. Qué es	2
2. Instalación y configuración	2
3. Conceptos	3
4. Escribir consultas	3
5. Ejecutar una búsqueda	4
6. El directorio de investigación	5
6.1 Índice	5
6.2 Traer registros desde fuera	5
6.3 Desde Zotero, Mendeley y EndNote	6
7. Informes	6
7.1 Una ejecución	6
7.2 Todo el directorio de investigación	6
7.3 Niveles	6
7.4 Formatos	7
7.5 Filtros	7
7.6 PRISMA	7
7.7 Sugerencias	8
7.8 Idiomas	8
8. Métricas de revistas	8
9. Web of Science	9
10. Logs y auditoría	9
11. Flujos de trabajo	9
12. Funciones y limitaciones	10
13. Pruebas	10
14. Licencia y conducta	11

[English](#) · [Português \(Brasil\)](#) · **Español** · [Deutsch](#) · [Français](#)

Traducción del manual en inglés, que es la referencia; los comandos, nombres de archivo, opciones y bloques de código se conservan como en el original.

1. Qué es

scitech-librarian es un instrumento reproducible de búsqueda bibliográfica para ciencia e ingeniería. Escribes una consulta estructurada una sola vez; la ejecuta contra hasta ocho bases de datos bibliográficas a través de sus APIs documentadas, archiva todo (registros, la cadena de consulta exacta enviada a cada base de datos, recuentos de resultados, un log) y escribe un informe de búsqueda bibliográfica con un diagrama de flujo PRISMA 2020. A lo largo de meses, las ejecuciones, más los registros que obtuviste por otros medios, se acumulan en un **directorio de investigación** que el mismo informe puede describir en conjunto — qué aportó cada búsqueda, qué contribuyó cada base de datos, cómo derivaron los recuentos, qué revistas importan.

Son cinco scripts de Python más dos módulos compartidos (`render.py`, `i18n.py`), sin dependencias más allá de la biblioteca estándar. No hay nada que instalar: copia los archivos, aporta tus claves, escribe `queries.json`, ejecuta.

Archivo	Función
<code>librarian.py</code>	ejecuta una búsqueda; archiva una ejecución; llama al informe
<code>project.py</code>	directorio de investigación: índice, ingesta de registros externos, estado
<code>report.py</code>	informes de una ejecución o de todo el directorio; PRISMA; filtros
<code>journals.py</code>	métricas de revistas (cifras del tipo factor de impacto) por año
<code>wos_manual.py</code>	Web of Science a mano (sin API gratuita utilizable)
<code>render.py</code>	renderizadores Markdown / HTML / LaTeX / texto y la cadena de PDF (importado por <code>report.py</code>)
<code>i18n.py</code>	idiomas del informe: el catálogo en / pt-BR / es / de / fr (importado por <code>report.py</code> ; §7.8)

Para agentes de IA. `AGENTS.md` en la raíz del repositorio es la descripción completa de la herramienta orientada a la máquina. Si trabajas con un agente de programación (Claude Code, Codex, Cursor...), dile: “*Lee AGENTS.md y luego haz una comprobación de novedad sobre X*” — contiene los comandos, los esquemas de archivos, los flujos de trabajo y las reglas que el agente no debe romper.

2. Instalación y configuración

Requisitos: Python 3.9 o más reciente. Opcional, para informes PDF tipografiados: una distribución LaTeX (xelatex, lualatex o pdflatex) o pandoc; sin ellos el PDF lo produce un escritor de texto plano integrado.

```
git clone https://github.com/fabiocampolim-design/scitech-librarian
cd scitech-librarian
cp .env.example .env # fill in what you have (or use the environment)
cp queries.example.json queries.json
python librarian.py --selftest
```

Las claves se leen del entorno del proceso. Copia `.env.example` a `.env` y rellena ese archivo, o define los mismos nombres de variable en tu shell, en la configuración de tu agente o lanzador, o como secretos de CI — `.env` sólo aporta lo que el entorno no haya definido ya, así que una variable definida fuera siempre gana y el archivo es opcional.

Claves:

Clave	Necesaria para	Cómo obtenerla
<code>CONTACT_EMAIL</code>	acceso al “polite pool” de OpenAlex/Crossref/Unpaywall	tu dirección

Clave	Necesaria para	Cómo obtenerla
ADS_TOKEN	NASA ADS	gratuito, https://ui.adsabs.harvard.edu/user/settings/token
SCOPUS_API_KEY	Scopus (+ red institucional/VPN)	gratuita, https://dev.elsevier.com/apikey/manage
SCOPUS_INSTTOKEN	Scopus sin VPN	pídelo a tu biblioteca
S2_API_KEY	Semantic Scholar más rápido	opcional
CORE_API_KEY	CORE (si está configurado en backends.json)	gratuita, https://core.ac.uk/services/api
WOS_STARTER_KEY	Web of Science Starter API (gramática restringida)	rara vez vale la pena

Cinco backends (OpenAlex, arXiv, INSPIRE-HEP, Semantic Scholar, Crossref) no necesitan clave ni institución. `python librarian.py --list` informa de qué claves se encontraron por cualquiera de las dos vías; `--selftest` demuestra que funcionan.

Uso integrado dentro de otro proyecto. Coloca los siete archivos en un subdirectorio `tools/`; `.env`, `queries.json` y `lit/` se buscan entonces en el directorio padre.

3. Conceptos

Bloque. Una consulta estructurada: una lista de grupos de sinónimos combinados con AND, cada grupo una lista de sinónimos combinados con OR. Un bloque tiene un nombre (A, CD, NOV...), un título y una nota. Los bloques viven en `queries.json`.

Ejecución. Una ejecución de `librarian.py`: cada bloque seleccionado contra cada backend seleccionado, archivada en `lit/runs/<timestamp>/`.

Directorio de investigación. Una carpeta (por defecto `lit/`, elige otra con `--outdir`) que contiene todas las ejecuciones de un proyecto, los registros ingeridos desde fuera, el índice del proyecto (`project.json`), las cifras del cribado PRISMA (`screening.json`), métricas de revistas, informes y logs de auditoría. Un directorio por proyecto; un laboratorio tiene varios.

Fuente manual. Registros que no vinieron de una ejecución: una exportación de Zotero o Mendeley, el archivo RIS de un colega, una sesión de Web of Science, una lista de referencias. Ingeridos con `project.py ingest`, conservan su procedencia (quién, cuándo, de dónde, método) y aparecen en cada informe como una fuente más, y en el flujo PRISMA en la columna correcta.

Registro. El esquema común que usa cada archivo: `title year doi journal authors url abstract cited_by issn block backend`. Los registros de proyecto fusionados llevan además `found_by` (qué fuentes lo encontraron) y `first_seen`.

Nivel. Cuánto contiene un informe: `simple` (unas pocas páginas), `intermediate` (cada registro único más análisis), `full` (todo, resúmenes incluidos — cientos de páginas para proyectos grandes).

4. Escribir consultas

`queries.json`:

```
{
  "PSC": {
    "title": "Perovskite solar cell degradation under humidity",
    "note": "grounding block -- expect thousands",
    "groups": [
      ["perovskite solar cell", "halide perovskite photovoltaic"],
      ["degradation", "stability"],
      ["humidity", "moisture"]
    ]
  }
}
```

```

},
"NOV": {
  "title": "novelty check: origami metamaterials as topological acoustic pumps",
  "note": "a SMALL number is the good outcome; read every hit",
  "groups": [["origami", "kirigami"], ["acoustic metamaterial", "phononic crystal"],
    ["topological pumping", "edge state"]],
  "arxiv_groups": [0, 2]
}
}

```

Reglas prácticas:

- No entrecomilles los términos; la herramienta los entrecomilla según la gramática de cada base de datos.
- Una palabra genérica sola (**model**, **structure**, **system**) en su propio grupo es la causa habitual de recuentos en las decenas de miles.
- **arxiv_groups** indica qué grupos (dos como máximo) recibe arXiv; arXiv se cuelga con booleanos profundamente anidados. Por defecto: los dos primeros. arXiv se pagina de 100 en 100 registros con una pausa de 3 s, así que un `--limit` grande es lento ahí.
- El bloque más informativo es una intersección deliberada de dos literaturas que sospechas que no se hablan. Un resultado cercano a cero es un hallazgo — *si* después lees cada resultado.
- Los operadores de proximidad (**NEAR/n**, **W/n**) no se pueden expresar; si tu artículo los necesita, conserva al lado cadenas escritas a mano para Web of Science / Scopus y cita esas.

5. Ejecutar una búsqueda

```

python librarian.py                # all blocks, every configured backend
python librarian.py --counts-only  # hit counts only (seconds)
python librarian.py --blocks NOV CD # selected blocks
python librarian.py --backends openalex ads
python librarian.py --skip arxiv   # drop a misbehaving backend
python librarian.py --limit 500    # records per block and backend (default 300)
python librarian.py --pdfs --pdf-blocks NOV # legal OA-PDF links via Unpaywall
python librarian.py --keep-junk    # keep non-curated venues (Zenodo, SSRN...)
python librarian.py --outdir lit_topomat # another research directory
python librarian.py --report-level intermediate --report-format md html pdf
python librarian.py --report-lang pt-BR # report in Portuguese (en, pt-BR, es, de, fr; §7.8)
python librarian.py --no-report
python librarian.py --queries other.json # another query file (default ./queries.json)
python librarian.py --backends-file b.json # another backends config; --init-backends writes the default
python librarian.py --timeout 60          # per-request timeout in seconds (default 45)
python librarian.py --list                # blocks and backend readiness, then exit
python librarian.py --selftest            # ping every backend, then exit

```

Lista completa de parámetros: `python librarian.py --help`. Cada opción tiene un valor por defecto; `--outdir`, `--verbose`, `--quiet` y `--log-dir` existen en todos los scripts.

Qué escribe una ejecución (`lit/runs/<stamp>/`):

Archivo	Contenido
<code>counts.json</code> , <code>counts.md</code>	recuentos de resultados por bloque y backend; tabla lista para pegar
<code>queries.json</code>	la cadena de consulta exacta enviada a cada backend
<code>blocks.json</code>	las definiciones de bloque usadas
<code>meta.json</code>	ajustes, backends y endpoints, versión, tiempos
<code>records/<block>_<backend>.json</code>	registros en bruto por backend (tras el filtro de revistas)

Archivo	Contenido
ris/<block>_<backend>.ris	RIS por bloque para Zotero/Mendeley/EndNote
all_records.json/.csv/.ris	desduplicados, ordenados por citas
all_records.bib, all_records.csl.json	el mismo conjunto como BibTeX y CSL-JSON
junk.json	registros eliminados por el filtro de revistas, con sus revistas
prisma.json	plantilla para las etapas manuales de PRISMA
run.log	todo lo impreso
report.*	el informe (véase §7)

Más `lit/counts_history.csv` (una fila por bloque/backend/ejecución, para la deriva) y `lit/logs/librarian_<stamp>_<pid>`. (log de auditoría: invocación, versiones, cada mensaje).

Los recuentos se guardan en un punto de control tras cada llamada a la API y Ctrl-C es seguro: un bloqueo al final de una ejecución larga no pierde nada.

6. El directorio de investigación

6.1 Índice

```
python project.py init --name "Topological materials review" --description "..."  
python project.py status
```

`status` lista cada miembro (ejecuciones y fuentes manuales) con fecha, número de registros, método y etiqueta, el estado de la carpeta de entrada y el último informe. Los miembros se descubren listando el directorio — no hay que declarar nada. `project.json` guarda solo lo que no se puede descubrir:

```
{"name": "...", "description": "...", "created": "2026-08-28",  
  "exclude": ["20260814T223331"],  
  "labels": {"20260828T095041": "August full scan"},  
  "block_aliases": {"X": "CD"},  
  "defaults": {"level": "simple", "format": ["md"], "metric": "openalex_2yr"}}
```

```
python project.py exclude 20260814T223331      # a test run you do not want in reports  
python project.py label 20260828T095041 "August full scan"  
python project.py alias X CD                   # block renamed between runs  
python project.py oa                           # Unpaywall pass over every member that lacks OA data  
python project.py oa --members 20260828T095041 # restrict the pass to these member ids
```

`oa` es la consulta de acceso abierto a posteriori: las ejecuciones hechas sin `--pdfs` y las fuentes manuales reciben los campos `is_oa` / `oa_pdf` (solo copias legales, en caché en `unpaywall_cache.json`), que las estadísticas de acceso abierto del informe y `--oa-only` cubren entonces para todo el proyecto.

6.2 Traer registros desde fuera

Tres maneras, todas terminan en `lit/manual/<name>/` con el archivo original, un `records.json` en el esquema común y un `source.json` con la procedencia:

1. **Línea de comandos** — la manera completamente descrita:

```
python project.py ingest export.ris --name zotero-aug --block CD \  
  --method citation --who "A. Colleague" --origin "Zotero group library" \  
  --note "reference lists of the three key papers"
```

Se pueden indicar varios archivos; `--kind` anula la detección por extensión (`ris`, `bibtex`, `csv`, `json`).

2. **Carpeta de entrada** — suelta archivos en `lit/inbox/` y ejecuta `python project.py ingest --inbox`; cada archivo se convierte en una fuente con su nombre (añade `--method` etc. para aplicarlo a todos).

3. **Web of Science** — `python wos_manual.py ingest` lee los archivos RIS que exportaste desde la interfaz de WoS y los registra como fuentes manuales con `method=database`.

Formatos aceptados: RIS (Zotero, Mendeley, EndNote, Web of Science, Scopus), BibTeX, CSV con fila de cabecera (nombres de columna de Scopus y WoS reconocidos; si no `title`, `year`, `doi`, `journal`, `authors`, `url`, `abstract`, `block`, `cited_by`) y listas de registros JSON (por ejemplo el `all_records.json` de la ejecución de un colega).

--method sigue las categorías de PRISMA 2020 para registros identificados por otros métodos: `database` (una exportación de base de datos — se une a la columna de bases de datos), `citation` (listas de referencias, artículos citantes), `website`, `organisation`, `expert` (la recomendación de un colega), `other`.

También puedes entregar archivos extra a un solo informe sin almacenarlos: `report.py --records file.ris`.

6.3 Desde Zotero, Mendeley y EndNote

Salida: cada ejecución escribe RIS (`all_records.ris`, `ris/` por bloque), BibTeX (`all_records.bib`) y CSL-JSON (`all_records.csl.json`); importa con File → Import. Los resúmenes, DOI y URL se conservan, y el nombre del bloque llega como palabra clave (`block:NOV`), de modo que los ítems importados ya vienen etiquetados.

Entrada: exporta una colección como RIS (Zotero: clic derecho → Export Collection → RIS; Mendeley: File → Export → RIS; EndNoteX: File → Export → RefMan RIS) e ingiérrela como arriba. No hay conexión en vivo con la API de Zotero (hoja de ruta).

7. Informes

7.1 Una ejecución

```
python report.py lit/runs/20260828T095041
python report.py --latest --level full --format html pdf
```

7.2 Todo el directorio de investigación

```
python report.py --project
python report.py --project --outdir lit_topomat --level intermediate --format md html
```

Los informes van a `lit/reports/<stamp>-<level>/`. El informe de proyecto añade una tabla de **Fuentes** (cada ejecución y fuente manual, su fecha, método, registros y “nuevos aquí” — los registros únicos que ninguna fuente anterior había encontrado), una **Línea de tiempo** (recuentos por bloque a lo largo de las ejecuciones; cuándo entraron los registros en el proyecto), un flujo PRISMA con ambas columnas de identificación y, cuando se ha ejecutado `journals.py`, métricas de revistas.

7.3 Niveles

Nivel	Secciones
<code>simple</code>	metadatos; fuentes; estrategia de búsqueda con la cadena exacta por backend; resumen de resultados; línea de tiempo; flujo PRISMA 2020 + lista PRISMA-S; 10 mejores registros por bloque; sugerencias
<code>intermediate</code>	+ cada registro único; solapamiento de fuentes (“encontrado solo aquí”); distribuciones por año / revista / autor; métricas de revistas; revistas filtradas; errores; estadísticas de acceso abierto; historial de recuentos

Nivel	Secciones
full	+ cada registro con resumen completo, lista de autores y qué fuentes lo encontraron; listas en bruto por fuente antes de la deduplicación; los registros filtrados; configuración de los backends; archivos project.json y source.json; el log de la ejecución; entorno

Tamaños, según el ejemplo distribuido (cuatro bloques, tres bases de datos CC0, 1.226 registros únicos): 6, 68 y 427 páginas de PDF.

7.4 Formatos

md (Markdown; se renderiza en GitHub), html (autocontenido, claro/oscuro, imprimible, diagrama SVG), tex (LaTeX con diagrama TikZ), pdf, txt (texto plano, diagrama ASCII). El PDF se compila desde el LaTeX con xelatex, lualatex o pdflatex si hay alguno instalado, si no con pandoc, si no con un escritor integrado que maquetará la versión en texto — la opción nunca falla.

7.5 Filtros

Opción	Efecto
--since DATE, --until DATE	conserva los miembros (ejecuciones / fuentes manuales) buscados en la ventana
--latest	solo el miembro más reciente (proyecto); la ejecución más nueva (modo simple)
--diff	conserva solo los registros <i>vistos por primera vez</i> dentro de la ventana — “qué aportaron las búsquedas desde DATE”
--year-from Y, --year-to Y	año de publicación
--backends a b	bases de datos / fuentes a incluir (las fuentes manuales son <code>manual:<name></code>)
--blocks A CD	bloques a incluir
--sources auto\ manual\ all	tipos de miembro
--records FILE...	RIS/BibTeX/CSV/JSON extra como fuente manual transitoria
--metric NAME --min-metric X	conserva los registros cuya métrica de revista es al menos X (véase §8)
--min-citations N	umbral de citas
--oa-only	solo registros con copia legal de acceso abierto (necesita datos de <code>--pdfs</code> o <code>project.py oa</code>)
--top N, --sort cited\ year\ metric	tamaño y orden de la tabla
--basename, --out	raíz del nombre de archivo y directorio de salida

Los filtros se listan en la tabla de metadatos del informe y en el ítem 9 de PRISMA-S, para que un informe filtrado nunca se confunda con la búsqueda completa.

7.6 PRISMA

El informe lleva un diagrama de flujo PRISMA 2020 (SVG en HTML, TikZ en LaTeX/PDF, ASCII en Markdown y texto) y una lista de verificación PRISMA-S para el informe de la búsqueda. La herramienta rellena lo que puede saber: registros identificados por base de datos (sumados sobre las ejecuciones en modo proyecto), registros identificados por otros métodos (fuentes manuales por método), registros recuperados, eliminados por automatización (el filtro de revistas), duplicados eliminados, registros pendientes de cribar. Es explícita en que *identificados* (lo que informa cada base de datos) y *recuperados* (lo que se descargó dentro de `--limit`) difieren.

Las etapas que solo un humano puede conocer se leen de `prisma.json` (ejecución única) o `screening.json` (directorio de investigación); en el primer informe se escribe una plantilla con valores `null`. Rellena los enteros a medida que avanza el cribado y vuelve a ejecutar el informe:

```
{"records_screened": 2216, "records_excluded": 2100,
 "reports_sought": 116, "reports_not_retrieved": 4, "reports_assessed": 112,
 "excluded_reasons": {"not_topological": 60, "no_experiment": 30},
 "other_sought": 12, "other_not_retrieved": 0, "other_assessed": 12,
 "other_excluded_reasons": {"duplicate_of_included": 3},
 "studies_included": 22, "reports_included": 31,
 "citation_searching": "reference lists of the 22 included studies",
 "prior_work": "none", "peer_review": "search strategy reviewed by the librarian"}
```

7.7 Sugerencias

Basadas en reglas, al final de cada informe: llamadas a backends fallidas, bloques con miles de resultados, bloques de tamaño de novedad (lee cada resultado), tope `--limit` alcanzado, una base de datos con alta proporción de revistas filtradas, ningún backend con calidad de cita, consulta de acceso abierto no ejecutada, etapas PRISMA sin rellenar, sin métricas de revistas, deriva de recuentos entre ejecuciones y — en modo proyecto — la ausencia de cualquier fuente manual.

7.8 Idiomas

```
python report.py --latest --lang pt-BR
python report.py --project --lang de --format pdf
python librarian.py --report-lang fr # the report written at the end of a run
```

`--lang` (`report.py`) y `--report-lang` (`librarian.py`) aceptan `en` (por defecto), `pt-BR`, `es`, `de` o `fr`; un directorio de investigación puede fijar su propio valor por defecto con `"defaults": {"lang": "es"}` en `project.json`, y una opción explícita prevalece sobre él. Solo cambia el texto propio del informe — títulos, cabeceras de tabla, las etapas PRISMA 2020 y el diagrama de flujo en todos los formatos, la lista PRISMA-S, los párrafos explicativos y las sugerencias — junto con el separador de millares del idioma. Lo que la herramienta encontró o recibió se reproduce exactamente como está, sea cual sea el idioma: títulos, resúmenes, autores y revistas de los registros, los nombres y notas de tus bloques, las cadenas de consulta exactas, los nombres de backend, las opciones citadas en el texto, los nombres de archivo, los volcados JSON y el log de la ejecución incrustado. La salida por consola, `run.log` y los logs de auditoría están siempre en inglés, de modo que las ejecuciones hechas en distintos idiomas siguen siendo consultables juntas.

8. Métricas de revistas

```
python journals.py fetch # every journal seen in the directory
python journals.py fetch --providers openalex --refresh
python journals.py import-scimago scimagojr_2024.csv --year 2024 [--all]
python journals.py import-jcr JCR_JournalResults_*.csv # Journal Citation Reports downloads
python journals.py import-csv other.csv --provider my_metric --year 2023 --name-col Journal --
value-col Value [--issn-col ISSN] [--delimiter ";"] # any name/value table; ISSN
python journals.py list --missing jcr_if # journals still to look up by hand
python journals.py show --metric scopus_citescore
```

Almacén: `lit/journals/metrics.json`, una entrada por revista indexada por ISSN (si no, por nombre normalizado), valores conservados **por año y nunca sobrescritos** — vuelve a obtenerlos el año que viene y el informe muestra la serie.

Proveedor	Clave	Métricas	Historial
openalex	ninguna	openalex_2yr (citación media a 2 años, una cifra del tipo factor de impacto), openalex_h, obras/citas por año	instantánea bajo el año de obtención
scopus	SCOPUS_API_KEY	scopus_citescore, sjr, snip	historial completo por año
scimago	ninguna; descarga el CSV del año de scimagojr.com	sjr, scimago_h, cuartil	un archivo por año
jcr	licencia	jcr_if	solo importación

El Journal Impact Factor (Clarivate JCR) es propietario: no hay API gratuita y la herramienta no le hará scraping. Los usuarios con licencia descargan CSV desde la página *Browse journals* del JCR (600 filas por descarga; corta por categoría y luego por cuartil) y los importan con `journals.py import-jcr FILE...` — las columnas y el año del JIF se detectan. `journals.py list --missing jcr_if` imprime las revistas de tu directorio todavía sin valor, que es la lista que hay que consultar. El protocolo completo está en `docs/JCR_IMPORT.md`. Para una métrica que cubra todas las revistas, el CSV de SCImago (~30.000 revistas, una descarga) es la vía práctica; `--all` importa el archivo entero, por defecto se importan solo las revistas vistas en tus registros.

En los informes: una columna de métrica en las tablas de registros, “revistas de este conjunto por métrica”, una tabla de evolución para revistas con dos o más años registrados, y el filtro `--min-metric`. `--metric` elige cuál (por defecto `openalex_2yr`, o `defaults.metric` en `project.json`).

9. Web of Science

La gramática completa `TS=/NEAR` está en la Expanded API, rara vez con licencia; el nivel gratuito Starter rechaza los booleanos complejos. Web of Science es, por tanto, un trabajo manual, hecho pequeño:

```
python wos_manual.py prep      # query files + CHECKLIST.md in WoS grammar
python wos_manual.py walk     # copies each query to the clipboard in turn
python wos_manual.py ingest   # RIS exports -> records, registered as manual sources
python wos_manual.py status
python wos_manual.py prep --queries other.json # a different query file (default ./queries.json)
```

La lista de verificación codifica los ajustes de la interfaz que rompen consultas silenciosamente (Core Collection, búsqueda Advanced, ediciones, forma etiquetada frente a desnuda).

10. Logs y auditoría

Cada script escribe `<outdir>/logs/<script>_<stamp>_<pid>.log` con la invocación exacta, las versiones de la herramienta y de Python, el directorio de investigación, cada aviso y error, y el resultado. La salida por consola es pequeña por defecto; `--verbose` muestra todo, `--quiet` solo avisos y errores; `--log-dir` mueve los logs. Las ejecuciones conservan además `run.log` (la transcripción de la consola) en el directorio de la ejecución.

11. Flujos de trabajo

Una comprobación de novedad (una tarde). Escribe 1–3 bloques de consulta cruzada; `--counts-only`; ajusta todo lo que esté en los miles; ejecución completa con `--pdfs`; lee las Sugerencias; lee cada resultado de los bloques pequeños a mano; anota lo que cribaste en `prisma.json`; vuelve a ejecutar `report.py`; importa el RIS en Zotero.

Una búsqueda sistemática sobre un proyecto (meses). `project.py init`. Vuelve a ejecutar `librarian.py` a intervalos con el mismo `queries.json`. Ingiere las sesiones de Web of Science y las exportaciones de los colegas.

`report.py --project` para la imagen general; `--project --since <último informe> --diff` para lo nuevo; `journals.py fetch` cada año. Rellena `screening.json` sobre la marcha; el diagrama PRISMA se completa solo, listo para el material suplementario.

Un laboratorio. Un directorio de investigación por proyecto (`--outdir`); cada uno tiene su propio índice, cribado e informes. Las carpetas de entrada permiten a los colaboradores soltar exportaciones sin aprender la herramienta. Deliberadamente no hay fusión entre proyectos: preguntas distintas, bloques distintos.

Un ejemplo resuelto. `docs/WALKTHROUGH.md` (en inglés) recorre un proyecto real desde `queries.json` hasta un diagrama PRISMA completo, con todos los comandos.

Con un agente de IA. Apúntalo a `AGENTS.md`; pídele que redacte `queries.json` a partir de tu pregunta de investigación, ejecute los barridos y recorra el informe contigo. El archivo de consulta estructurado, los archivos JSON y el informe se diseñaron para que un agente los escriba y audite.

12. Funciones y limitaciones

Funciones: una consulta estructural traducida a ocho gramáticas nativas; bases de datos como configuración JSON (`--init-backends`); ejecuciones archivadas y citables con cadenas de consulta exactas e historial de recuentos; puntos de control y Ctrl-C seguro; un filtro de revistas con justificantes; cinco backends sin clave; NASA ADS e INSPIRE para física; enlaces legales a PDF de acceso abierto vía Unpaywall; informes de tres niveles en cinco formatos con PRISMA 2020 y PRISMA-S; directorios de investigación con fuentes manuales, procedencia, línea de tiempo e informes diferenciales; métricas de revistas con serie por año; logs de auditoría; una suite de pruebas sin conexión (301 comprobaciones) y CI.

Limitaciones, todas por diseño o por el mundo:

- Los recuentos no son comparables entre bases de datos; los operadores de proximidad se descartan. Descubre aquí; cita una base de datos en el artículo.
- Los resultados de Scopus requieren derecho institucional (red/VPN). La API de Web of Science rara vez tiene licencia; usa la vía manual.
- arXiv recibe como máximo dos grupos por bloque.
- `--limit` limita los registros por bloque y backend (los más citados primero); los bloques grandes son una porción, no el conjunto completo. Súbelo cuando necesites completitud.
- OpenAlex indexa repositorios no curados (~15 % de sus registros); filtrados por defecto, conservados en `junk.json`.
- Sin descarga de PDF (solo enlaces de Unpaywall), sin bola de nieve, sin grafo de citas, sin conexión en vivo con Zotero/Mendeley (hoja de ruta); BibTeX y CSL-JSON se escriben, no se leen de vuelta desde una biblioteca Zotero.
- Métricas de revistas: los valores de OpenAlex son instantáneas; el factor de impacto del JCR es propietario y solo importable; emparejar revistas por nombre es imperfecto cuando un registro no tiene ISSN.
- La deduplicación es por DOI, si no por los primeros 90 caracteres del título; los pares preprint/publicado con títulos distintos sobreviven como dos registros.
- Google Scholar no es ni será un backend (sin API; el scraping viola sus términos).

13. Pruebas

`python tests/test_librarian.py`

Sin conexión, solo biblioteca estándar, sin claves: los backends se ejecutan contra respuestas de API grabadas, el generador de informes contra ejecuciones y directorios de investigación sintéticos, y la línea de comandos de cada script se ejercita de extremo a extremo. El archivo es también un módulo de pytest (`pytest tests/`). La CI ejecuta pyflakes y la suite en Linux, Windows y macOS bajo Python 3.9 y 3.13.

14. Licencia y conducta

Apache License 2.0. La herramienta está hecha para que respetar los términos de servicio de cada base de datos sea el camino fácil: solo APIs documentadas, límites de tasa respetados, una dirección de contacto en cada petición, sin scraping, sin elusión de muros de pago.